



Facultad de ingeniería

Docente

Leidy Mariani Dorado Flórez

Materia

Sistemas Distribuidos

Trabajo

Avance - 3: Documento final

Estudiantes

Heidy Gabriela Jalvin Avirama
Ricardo Adolfo Hoyos
Juan Rodriguez
Jhonn Alexander Pantoja Jimenez

Popayán, Cauca 26 de mayo de 2026

1. DESCRIPCIÓN GENERAL DEL SISTEMA

Propósito del Sistema

El sistema tiene como propósito ofrecer una plataforma tecnológica para la administración de citas médicas, garantizando alta disponibilidad, resiliencia y escalabilidad. Se diseñó bajo una arquitectura distribuida que permite gestionar el ciclo de vida completo de la atención al paciente, desde el registro de usuarios y agendamiento, hasta el registro del historial clínico y el envío automatizado de notificaciones.

Problemática que resuelve

El problema que resolvemos es que pedir una cita médica suele ser complicado: hay que llamar, esperar mucho tiempo en la línea o hacer largas filas, para al final escuchar que no hay disponibilidad en la fecha que necesitas.

Nuestra plataforma facilita todo ese proceso. El usuario puede agendar su cita con el doctor en el horario que prefiera o en el que esté disponible, sin esperas innecesarias. Además, podrá llevar un registro fácil de sus citas y recibir notificaciones oportunas, todo en un solo lugar y de manera práctica.

Funcionalidades principales

1. **Gestión de autenticación:** Registro, inicio de sesión seguro y verificación de roles (Paciente/Doctor).
2. **Gestión de citas:** Consulta de disponibilidad horaria, agendamiento, cancelación y reprogramación de citas médicas.
3. **Historial Clínico:** Registro de notas médicas y trazabilidad cronológica de los eventos asociados a cada paciente.
4. **Notificaciones Automatizadas:** Generación de alertas sobre cambios de estado en las citas y gestión de buzón de lectura por paciente.
5. **Enrutamiento:** Centralización de peticiones y tolerancia a fallos mediante un API Gateway con patrón Circuit Breaker.

Usuarios del sistema

- **Pacientes:** Usuarios que utilizan el sistema para registrarse, agendar citas, consultar su historial, recibir y visualizar notificaciones.
- **Doctores / Personal Médico:** Usuarios con permisos específicos validados por el sistema para agregar notas manuales al historial clínico.

Alcance actual del proyecto

El proyecto ha alcanzado su etapa funcional completa (100% de los requisitos del corte). Actualmente, el sistema opera sobre una arquitectura contenerizada en Docker que incluye:

- Un API Gateway como punto de entrada único.
- Cuatro (4) microservicios funcionales (Autenticación, Citas, Historial y Notificaciones).
- Cuatro (4) bases de datos MySQL totalmente desacopladas (una por servicio).
- Un Message Broker (RabbitMQ) que habilita una arquitectura orientada a eventos, permitiendo procesamiento asíncrono para historiales y notificaciones sin afectar el rendimiento de las operaciones principales.

2. ARQUITECTURA FINAL DEL SISTEMA

Microservicios Implementados y Bases de Datos Desacopladas

El sistema se compone de cuatro microservicios independientes, cada uno encapsulado en su propio contenedor y conectado exclusivamente a su propia base de datos relacional:

- **Servicio-Autenticación:** Responsable de la gestión de identidades y roles.
- **Persistencia:** Base de datos db_autenticacion (MySQL).
- **Servicio-Citas:** Núcleo del negocio, encargado de la disponibilidad y agendamiento.
- **Persistencia:** Base de datos citas_db (MySQL).
- **Servicio-Historial:** Registra la trazabilidad y las notas médicas de cada cita.

- **Persistencia:** Base de datos historial_db (MySQL).
- **Servicio-Notificaciones:** Gestiona los mensajes y alertas para los pacientes.
- **Persistencia:** Base de datos notificaciones_db (MySQL).

API Gateway (Punto de Entrada Centralizado)

Se implementó un Servicio-gateway desarrollado en Flask que actúa como puente de entrada al usuario. Sus dependencias y responsabilidades son:

- Enrutamiento estático hacia la red interna de microservicios.
- Aislamiento del cliente frente a la complejidad de la infraestructura.
- Implementación del patrón Circuit Breaker más half-open para la tolerancia a fallos.
- Monitoreo de latencia y estado de salud (/health) de todos los nodos.

Puertos y Conexiones (Topología de Red)

Todos los contenedores están interconectados mediante una red privada de Docker (red_citas_medicas). Los servicios exponen el puerto 5000 internamente. Por motivos de desarrollo y administración, se han mapeado los puertos al Host de la siguiente manera:

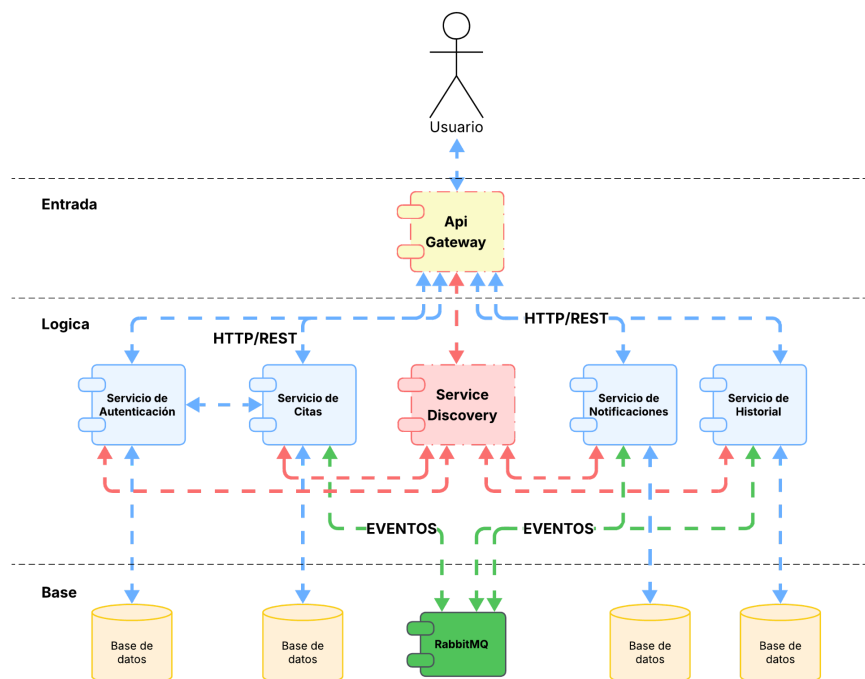
Servicio	Puerto Interno	Puerto Expuesto	Base de Datos	Puerto de la BD
GATEWAY	5000	5000		
AUTENTICACIÓN	5000	5001	db_autenticacion	3307
CITAS	5000	5002	db_citas	3308
HISTORIAL	5000	5003	db_historial	3309
NOTIFICACIONES	5000	5004	db_notificaciones	3310
RABBIT MQ	5672	5672		

Comunicación entre Servicios y Flujo de Peticiones

El sistema implementa dos flujos de comunicación inter-servicio dependiendo de la criticidad de la petición:

- **Comunicación Síncrona (HTTP/REST):** Utilizada para procesos dependientes e inmediatos. Cuando un usuario consulta la disponibilidad de un doctor mediante el Gateway, este reenvía la petición al Servicio-Citas. Antes de procesar la respuesta, el Servicio-Citas realiza una petición HTTP GET interna (**`http://autenticacion:5000/usuarios/<id>`**) para validar que el identificador corresponda realmente a un rol "Doctor".
- **Comunicación Asíncrona (Arquitectura Orientada a Eventos):** Utilizada para procesos secundarios que no deben bloquear al usuario. Se implementó **RabbitMQ**. Cuando el Servicio-Citas confirma un agendamiento, no llama directamente a Historial ni a Notificaciones; en su lugar, publica un evento (**ej. CITA_CREADA**) en las colas eventos_citas y eventos_notificaciones. Los servicios de Historial y Notificaciones ejecutan hilos o canales secundarios que escuchan y procesan estas colas de manera autónoma.

Diagrama Actualizado del Sistema



3. JUSTIFICACIÓN DE ARQUITECTURA

Criterios de división de microservicios

La arquitectura del sistema se diseñó pensando desde el inicio en ser **distribuido**.

Por eso se organizó en diferentes servicios, cada uno con responsabilidades específicas:

- **Autenticación:** Maneja la seguridad, credenciales y verificación de roles.
- **Citas:** Contiene la lógica principal para agendar y validar horarios médicos.
- **Historial y Notificaciones:** Brindan soporte y trazabilidad, evitando que tareas pesadas como reportes o alertas afecten el rendimiento del agendamiento.

La razón principal de esta separación es que el agendamiento de citas médicas es un proceso crítico. El sistema debe seguir funcionando incluso si alguno de los servicios falla. Por eso no puede ni debe ser un **monolito**: la resiliencia y disponibilidad dependen de que cada servicio esté desacoplado y pueda operar de manera independiente.

Ventajas Técnicas Obtenidas

- **Aislamiento de fallos:** Si la base de datos de notificaciones se corrompe o satura, el agendamiento de citas médicas no se ve interrumpido.
- **Procesamiento Asíncrono (Desacoplamiento):** Gracias a la inclusión de RabbitMQ, la comunicación dejó de ser estrictamente bloqueante. El servicio de citas ahora delega tareas publicando eventos, liberando rápidamente la conexión para el usuario.
- **Independencia Tecnológica y de Despliegue:** Cada módulo tiene su propio ciclo de vida y puede reiniciarse o actualizarse sin afectar a los demás.

4. DIFICULTADES ENCONTRADAS Y DECISIONES TÉCNICAS

Durante el desarrollo e implementación, se superaron los siguientes retos técnicos:

- **Condiciones de Carrera en el Arranque (Race Conditions):** Los microservicios intentaban conectarse a RabbitMQ o a las bases de datos antes de que estos contenedores estuvieran listos.
 - **Solución:** Se implementó un algoritmo de retroceso y reintento en el código (ej. `for i in range(100): time.sleep(3)`) ejecutado en hilos secundarios para no bloquear el inicio del servidor Flask.
- **Visibilidad de Logs en Contenedores:** Al ejecutar Docker en modo detached, las impresiones de consola no aparecían en tiempo real debido al búfer de Python.
 - **Solución:** Se forzó el parámetro `flush=True` en todas las instrucciones `print()` del sistema para asegurar la auditoría en tiempo real.
- **Resolución de Nombres en Redes Docker:** Al inicio, los servicios intentaban comunicarse apuntando a localhost, lo cual fallaba dentro de los contenedores.
 - **Solución:** Se unificaron los puertos de escucha a 5000 y se utilizó el DNS interno de Docker, apuntando directamente a los nombres de los servicios (ej. <http://autenticacion:5000/usuarios/<id>>).

5. TOLERANCIA A FALLOS Y MONITOREO

El sistema está diseñado para ser altamente resiliente, implementando salvaguardas tanto en la comunicación síncrona como en la asíncrona.

Manejo de Errores y Validaciones

- **Bloques de Excepción (try/except/finally):** Todas las consultas a las bases de datos y conexiones externas están encapsuladas. Los cursores y conexiones se cierran de forma segura en el bloque finally para evitar fugas de memoria.
- **Estandarización de Respuestas HTTP:** El sistema no expone los errores crudos del servidor. Los errores de validación retornan 400 Bad Request, credenciales erróneas 401 Unauthorized, recursos no encontrados 404 Not Found y caídas internas 500 Internal Server Error con JSON estructurados (`{"Error": "mensaje"}`).

Circuit Breaker

Para proteger los recursos ante caídas en cascada, el API Gateway centraliza una máquina de estados del patrón Circuit Breaker. Cada servicio tiene su propio circuito independiente gestionado en un diccionario en memoria.

- **Cerrado (CLOSED):** Flujo normal. Se configuran timeouts estrictos de 3 segundos en las peticiones HTTP con la librería requests.
- **Abierto (OPEN):** Si un servicio acumula 3 fallos consecutivos (por timeout o error de red), el circuito se abre. El Gateway intercepta las peticiones y responde instantáneamente con un HTTP 503 Service Unavailable, evitando sobrecargar el servicio caído.
- **Recuperación (HALF_OPEN):** Al transcurrir 10 segundos de apertura, el circuito pasa a Half-Open. Permite el paso de una única petición de prueba; si es exitosa, reinicia los fallos a 0 y cierra el circuito. Si falla, lo reabre inmediatamente.

Resiliencia Asíncrona (Recuperación de Servicios)

Para la mensajería, se implementaron mecanismos que garantizan Zero Data Loss (Cero pérdida de datos):

- **Colas Durables:** RabbitMQ declara las colas con el parámetro durable=True. Si un nodo se apaga abruptamente, los eventos no procesados se guardan en el disco.
- **Acks Manuales:** El procesamiento de eventos en Historial y Notificaciones requiere una confirmación manual (ch.basic_ack). Si el servicio se cae a mitad del guardado en base de datos, el mensaje no se pierde y es reencolado.

Monitoreo y Logs (Health Checks)

Se integraron múltiples capas de observabilidad:

- **Health Checks Individuales:** Cada microservicio expone un endpoint /health (ej. {"status": "ok", "servicio": "Historial"}).
- **Tablero de Monitoreo Centralizado:** El API Gateway expone el endpoint GET /monitoreo, el cual escanea iterativamente todos los microservicios.

- **Auditoría:** La ruta /monitoreo retorna en tiempo real un JSON detallado con la latencia exacta de respuesta (ej. "tiempo_respuesta": "0.0150s"), el código HTTP de respuesta, el contador actual de fallos y el estado en vivo del Circuit Breaker (cerrado, Abierto, HALF_OPEN).

6. ESCALABILIDAD Y ADMINISTRACIÓN

Aunque la arquitectura actual basada en microservicios sienta las bases para un sistema altamente escalable, su despliegue actual en un entorno local presenta ciertas limitaciones propias de la fase de desarrollo.

Limitaciones Actuales

- **Orquestación en Nodo Único:** Actualmente, el sistema se ejecuta bajo Docker Compose en una sola máquina virtual o servidor físico. Esto impide el escalado horizontal dinámico (crear réplicas de contenedores en tiempo real según la demanda).
- **Puntos Únicos de Fallo (SPOF) en la Infraestructura:** Aunque los microservicios están desacoplados, componentes de infraestructura como el API Gateway, el broker RabbitMQ y las bases de datos operan como instancias únicas. Si el contenedor del Gateway cae por completo, el sistema entero queda inaccesible.
- **Sobrecarga de Validación Interna:** En el flujo actual, el Servicio de Citas debe hacer una petición HTTP al Servicio de Autenticación para validar el rol del doctor antes de procesar la disponibilidad. En un escenario de alto tráfico, esto podría generar un cuello de botella en el servicio de autenticación.

Posibles Mejoras Futuras

Para llevar el sistema a un entorno de producción de alta demanda, se proponen las siguientes evoluciones arquitectónicas:

- **Migración a Kubernetes:** Reemplazar Docker Compose por Kubernetes para implementar autoescalado horizontal. Esto permitiría, por ejemplo,

instanciar automáticamente 5 réplicas del Servicio de Citas durante las horas pico de agendamiento y reducirlas a 1 en la noche.

- **Implementación de JWT (JSON Web Tokens):** En lugar de validar el rol mediante peticiones HTTP internas, el Servicio de Autenticación emitiría un token JWT cifrado. El API Gateway validaría la firma del token directamente en la frontera, liberando a los microservicios internos de tareas de validación y reduciendo la latencia de red.

7. SEGURIDAD BÁSICA

La seguridad del sistema se abordó desde la protección de la infraestructura y el resguardo de la información sensible.

Manejo de Variables de Entorno y Credenciales

El principio fundamental aplicado fue la separación estricta entre el código fuente y las configuraciones.

- **Archivos .env:** Ninguna credencial de base de datos, puerto o clave secreta fue escrita en texto plano (hardcoded) dentro de los scripts de Python. Todos los parámetros sensibles se consumen mediante la librería `os` de Python
 - (ej. `os.getenv("HISTORIAL_DB_PASSWORD")`).
- **Inyección mediante Docker Compose:** Las variables se inyectan dinámicamente a los contenedores a través de la instrucción `env_file` o la etiqueta `environment` en el archivo `docker-compose.yml`.
- **Protección del Repositorio (.env.example):** Para cumplir con las buenas prácticas de Git, el archivo `.env` real fue excluido del control de versiones mediante `.gitignore`. En su lugar, el repositorio cuenta con un archivo `.env.example` que contiene la estructura requerida (con valores vacíos o ficticios) para que cualquier desarrollador pueda levantar el entorno de manera segura.

- **Aislamiento de Red (Docker Networks)**

La arquitectura implementa seguridad a nivel de red interna. Los contenedores de microservicios y bases de datos se comunican a través de una red privada virtual de Docker (`red_citas_medicas`). Al exterior, el único componente que debería estar expuesto en un entorno de producción real es el puerto 5000 del API Gateway, actuando como escudo protector que oculta la topología interna, direcciones IP y puertos reales de las bases de datos ante posibles atacantes.